TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

# Week 14

Continue.dev: configure, chat, autocomplete

Phase 3 — AI-Assisted Development · Weeks 13–18 · Modules 05–06 + Capstone

**No D2L login required.** This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

## In this packet


1. 05.04 Install + Configure Continue.dev
2. 05.05 Chat
3. 05.06 Autocomplete

**Learning objective:** Install Continue.dev in VS Code and write `~/.continue/config.yaml` so the 70B serves chat, edit, and apply while the 8B serves autocomplete, then verify with a first chat and a ghost-text completion.

# Lesson 04: Install + Configure Continue.dev

**By the end, Continue.dev is installed in VS Code, configured to talk to the college's shared vLLM endpoint, and you've sent your first chat message through the IDE.**

~25 minutes of clicking + typing

---

## Step 1: install the extension

**1.** Open VS Code. Open the extensions side bar ( `Ctrl+Shift+X` ).

**2.** Search for **Continue**. The extension you want is published by **Continue** (the company), id `continue.continue`.

**3.** Click **Install**. (If you already installed it in lesson 00, confirm it shows **Installed**.)

**4.** After install, look at the activity bar (left edge). There's a new icon — a small Continue logo. That's the Continue side bar. Click it.

**5.** First-launch prompt: Continue may offer "Try it for free" with their hosted models. **Click "Use my own models"** or "Skip" — we point it at the college endpoint, not a hosted service.

**6.** The Continue chat panel opens. At the bottom there's a model selector. It probably says "No models configured." That's fine — we'll fix it next.

---

## Step 2: open the config

**7.** Continue's config lives at `~/.continue/config.yaml`. Open the integrated terminal in VS Code.

```
ls -la ~/.continue/
```

You'll see something like:

```
config.yaml      (or config.json on older versions)
docs/
index/
sessions/
```

**8.** Open the config in VS Code.

```
code ~/.continue/config.yaml 2>/dev/null || code ~/.continue/config.json
```

(Older Continue versions still use JSON. The lesson assumes YAML; if you got JSON, follow the JSON-version note at the end.)

---

## Step 3: write the config

**9.** Replace the entire contents of `~/.continue/config.yaml` with the following. (If your instructor gave you the Tailscale IP, substitute it in both `apiBase` lines. If no key is required on the campus network, you may delete both `apiKey` lines.)

```
name: College Coder (vLLM)
version: 1.0.0
schema: v1

models:
  - name: Llama 3.3 70B (chat/edit/agent)
    provider: openai                # vLLM is OpenAI-compatible
    model: meta-llama/Llama-3.3-70B-Instruct
    apiBase: http://gpu.college.local:8000/v1
    apiKey: sk-college-shared       # shared key; or omit on the campus network
    roles:
      - chat
      - edit
      - apply

  - name: Llama 3.1 8B (autocomplete)
    provider: openai
    model: meta-llama/Llama-3.1-8B-Instruct
    apiBase: http://gpu.college.local:8000/v1
    apiKey: sk-college-shared
    roles:
      - autocomplete

context:
  - provider: code
  - provider: diff
  - provider: terminal
  - provider: problems
  - provider: folder
  - provider: codebase
  - provider: file
  - provider: docs
```

**10.** Save (`Ctrl+S`).

**11.** Read the config you just wrote:

- **`models:`** — a list of model definitions.
- Each has a friendly `name` (shown in the picker), a `provider` (`openai` here — vLLM speaks the OpenAI API), the `model` (the exact vLLM model id from lesson 02), an `apiBase` (the shared endpoint), an `apiKey` (the shared key), and `roles` (which Continue features it can serve).
- The **70B** model handles **chat** (the conversation panel), **edit** (`Ctrl+I` inline edits), and **apply** (turning chat suggestions into actual file edits).
- The **8B** model handles **autocomplete** only.
- **`context:`** — providers that let you reference things in chat with `@`. We'll use these in lesson 05.

## Step 4: reload Continue

**12.** Continue picks up config changes automatically, but a clean reload is the safest. In the command palette (`Ctrl+Shift+P`), run **"Continue: Reload"** or just close and reopen the Continue side bar.

**13.** Look at the bottom of the Continue chat panel. The model selector should now show **"Llama 3.3 70B (chat/edit/agent)"**. If it still says "No models," your YAML has a syntax error — re-check indentation (2 spaces, no tabs) and try again.

---

## Step 5: first chat

**14.** Type into the Continue chat box:

```
Reply with the single word: hello
```

**15.** Press Enter (or click send). After a few seconds, the response should stream back: `hello` (or `Hello`).

**16. You're talking to the college's 70B.** The request went over the network to the shared vLLM box and back — no model is running on your laptop.

**17.** Try a coding question.

```
In one sentence, what does the @app.get decorator do in FastAPI?
```

You should get a focused, accurate answer.

---

## Step 6: confirm autocomplete is loaded

**18.** Open a Python file from your repo, e.g. `~/docker-course/fastapi-app/main.py`.

**19.** At the bottom of `main.py`, hit Enter to start a new line and type:

```
def add(a: int, b: int) -> int:
```

**20.** Pause. After a brief delay, you should see **ghost text** appear suggesting `return a + b`. Press **Tab** to accept, or Esc to dismiss. This suggestion came from the **8B** model — notice it's near-instant compared to the chat response.

**21.** If no ghost text appears: open the Continue side bar, look at the bottom — there's a small status line. If it mentions the autocomplete model, that's the issue. Re-check the `roles: [autocomplete]` block on the 8B in

your config.

**22. Don't keep the change.** Undo (`Ctrl+Z`) — we'll work the codebase properly later.

---

## Step 7: a note on remote / SSH setups

**23.** If your VS Code is attached to a remote host via **Remote — SSH** or **Remote — WSL**:

- `~/.continue/config.yaml` lives on the machine where the Continue extension host runs — i.e. on the **remote** when you're attached via Remote — SSH. Edit it there (the `code ~/.continue/config.yaml` command in step 8 opens the right one).
- The `apiBase` is the **same college endpoint** regardless of where your editor runs: `http://gpu.college.local:8000/v1`. There's no port forwarding to set up — you're calling a shared network service, not a local process.
- Whichever machine runs the extension host just needs to reach the endpoint. Confirm with the lesson 02 `curl` from that machine if chat fails.

**24.** Log it.

```
echo "Continue.dev configured to talk to the college vLLM endpoint" >> ~/docker-course/notes/notes.txt
```

---

## Watch out

- **YAML indentation.** 2 spaces per level, no tabs. Continue is picky.
- **`provider: openai`, not a vLLM-specific provider.** vLLM implements the OpenAI API, so Continue's `openai` provider is correct. The `apiBase` is what points it at the college box.
- **`apiBase` ends in `/v1`** and is `http://`, not `https://` — plain HTTP because it's an internal campus service.
- **Roles matter.** Wrong role on the wrong model means autocomplete uses the slow 70B (laggy on every keystroke), or chat uses the smaller 8B (weaker answers).
- **Key mismatch = "Failed to fetch" or 401.** If chat errors immediately, confirm `apiKey` matches the shared key (or that you removed it, if your network needs none).
- **Older Continue versions** use `~/.continue/config.json` instead of YAML. The same shape applies — JSON syntax, same keys. If you have JSON, port the YAML above by hand.

---

## Checkpoint

```
test -f ~/.continue/config.yaml -o -f ~/.continue/config.json && echo "OK: config present"
```

Should print `OK: config present`.

In the Continue chat panel:

- The model picker shows **Llama 3.3 70B (chat/edit/agent)**.
- Sending "Reply with the single word: ok" returns `ok` within a few seconds.

In any Python file, typing a function signature followed by a newline triggers ghost-text autocomplete (from the 8B).

---

**Next up:** 05-chat.md — Chat with the model about your repo

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

**Learning objective:** Use the `@file`, `@codebase`, `@diff`, and `@terminal` context providers to bring real repository context into Continue chat and get grounded answers about your FastAPI code.

# Lesson 05: Chat

**By the end, you will have used `@file`, `@codebase`, `@diff`, and `@terminal` to bring real context from your FastAPI repo into a chat.**

~20 minutes of typing + reading

> Plain "explain this" chat is fine, but the magic is **context**. Continue's `@` providers pull files, diffs, terminal output, and whole codebases into the prompt before the model sees it.

---

**1.** In VS Code, open the FastAPI project.

```
cd ~/docker-course/fastapi-app
```

Then `File → Open Folder…` and pick `~/docker-course/fastapi-app`.

**2.** Open the Continue side bar (Continue icon in the activity bar). Confirm the model picker shows **Llama 3.3 70B (chat/edit/agent)**.

---

## Drill 1: a vanilla question

**3.** In the chat box, type:

```
Explain what FastAPI's @app.post decorator does and how it differs from @app.get.
```

Send. Read the response. Note the latency — first token in a few seconds, then a stream.

---

## Drill 2: ask about a specific file

**4.** Type `@` in the chat box. A picker appears. Type `file` and select **File**.

**5.** A file-picker opens. Pick `main.py` from your repo.

**6.** After the `@main.py` chip appears in the input, type:

```
Walk me through what main.py does, line by line.
```

Send.

**7.** Read the response. The model now has the actual contents of `main.py` and references real lines. **This is what beats googling — the model sees your code.**

---

## Drill 3: ask about a router

**8.** New chat (click the `+` icon at the top of the Continue panel — fresh context).

**9.** Reference `routers/items.py`:

```
@routers/items.py
What HTTP methods does this router expose, and what does each one do?
```

Send. Read.

---

## Drill 4: `@codebase` — full repo context

**10.** New chat.

**11.** Type:

```
@codebase
Where in this project does the database connection get created? Reference the file
and line if possible.
```

Send.

**12.** `@codebase` triggers Continue to **embed and search** your whole repo (it builds an index in the background; the first use after opening a folder takes a few seconds). The model gets the most relevant chunks, then answers.

**13.** Read the answer. It should mention `database.py` and the `engine = create_engine(...)` line.

---

## Drill 5: `@diff` — what's changed since the last commit

**14.** Make a small uncommitted change. Open `main.py` and add a single comment at the top:

```
# This is a Module 05 chat-context test
```

**15.** Save. **Don't commit.**

**16.** New chat in Continue.

```
@diff
Summarize the change I'm about to commit.
```

**17.** The model receives the `git diff` output and summarizes. Useful for writing commit messages.

**18.** Undo the change before moving on (`Ctrl+Z`, save).

---

## Drill 6: `@terminal` — pull recent terminal output

**19.** In VS Code's terminal, run:

```
docker compose up -d 2>&1 | head -n 5
```

**20.** New Continue chat.

```
@terminal
What did Docker Compose just do?
```

**21.** The model sees your terminal scrollback and explains. Useful for "what does this error mean."

**22.** Force an error to feel this in anger:

```
docker run --not-a-real-flag
```

**23.** New chat.

```
@terminal
What's wrong with the last command I ran?
```

Read the answer — it should explain the flag is invalid and suggest `docker run --help`.

## Drill 7: combine providers

**24.** New chat.

```
@file:routers/items.py @file:routers/users.py
The items router has a class for input/output Pydantic models. The users router has
a similar pattern. Compare the two and tell me whether they're consistent.
```

**25.** The model gets both files in context and compares. This is faster than reading both yourself.

---

## Drill 8: ask for a code suggestion (don't apply yet)

**26.** New chat.

```
@file:routers/items.py
Write a `PUT /items/{item_id}` route that updates an existing item. Return 404 if
not found. Use the same pattern as `delete_item`.
```

**27.** The model proposes code. Read it carefully — does it look like the existing patterns? Does it use the same `models`, `db`, `HTTPException`?

**28. Don't click "Apply" yet** — we'll cover apply/edit in lesson 07. For now, just read the suggestion and verify it looks reasonable. Keep it open for lesson 07.

---

**29.** Log it.

```
echo "Chat drilled: @file, @codebase, @diff, @terminal" >> ~/docker-
course/notes/notes.txt
```

---

## Watch out

- **Context is finite.** Llama 3.3 70B has a large context window, but it's still finite — and the endpoint is shared, so smaller prompts run faster. `@codebase` on a huge repo can still lose detail. Use `@file` for surgical questions, `@codebase` only when you're searching.
- **The model can be wrong.** It will sometimes invent function names or APIs. Read the response, don't trust blindly. If you're going to apply changes (lesson 07), read the diff first.

- **New chat for new topics.** Continue keeps conversation memory in the chat session. Switching topics? `+` icon for a fresh chat. Stale context confuses the model.
- **Don't paste secrets.** Anything you put in the chat goes into the model's prompt. With the college endpoint that prompt goes to a shared campus server (not a third-party cloud) — but get the hygiene habit anyway.

---

## Checkpoint

In Continue, you have:

- [ ] Sent a chat message and received a response.
- [ ] Used `@file` to bring a specific file into context.
- [ ] Used `@codebase` to ask a search-flavored question.
- [ ] Used `@diff` to summarize uncommitted changes.
- [ ] Used `@terminal` to ask about recent terminal output.

---

**Next up:** 06-autocomplete.md — Tab-completing as you type

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

**Learning objective:** Write a working `PUT /items/{item_id}` route using tab autocomplete, reading each ghost-text suggestion before accepting with Tab, refining by typing, or dismissing with Esc.

# Lesson 06: Autocomplete

**By the end, you will have used tab autocomplete to write a real route in your FastAPI app.**

~15 minutes of typing

> Autocomplete is the most-used Continue feature. As you type, the small 8B model predicts what comes next as **ghost text**. Tab to accept, keep typing to refine, Esc to dismiss.

---

**1.** Make sure your FastAPI repo is open in VS Code and the venv is **not** active in your terminal (autocomplete works regardless — keeping the prompt clean).

**2.** Open `routers/items.py`.

**3.** Find the `delete_item` function. Position your caret at the end of that function's last line.

---

## Drill 1: complete a route

**4.** Press Enter twice to start a new function. Type:

```
@router.put("/{item_id}", response_model=ItemOut)
```

**5.** Press Enter. Pause. Ghost text should appear suggesting a function signature.

**6.** Read the suggestion before pressing Tab. Does it look like:

```
def update_item(item_id: int, item: ItemIn, db: Session = Depends(get_db)):
```

or something close? If yes, **Tab** to accept.

**7.** Press Enter. The model should now suggest the body — query, 404 check, mutation, commit, return.

**8.** Read each suggestion before accepting. Tab if it looks right; keep typing if you want to override.

**9.** End of the function — your file should have something like:

```
@router.put("/{item_id}", response_model=ItemOut)
def update_item(item_id: int, item: ItemIn, db: Session = Depends(get_db)):
    db_item = db.query(models.Item).filter(models.Item.id == item_id).first()
    if db_item is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Item not
found")
    for key, value in item.model_dump().items():
        setattr(db_item, key, value)
    db.commit()
    db.refresh(db_item)
    return db_item
```

(Yours may differ in details — that's fine. Read it for correctness.)

**10.** Save.

---

## Drill 2: feel partial vs full accept

**11.** In the same file, position at the bottom and start typing a docstring for `update_item`:

```
"""
```

**12.** Ghost text appears with a full docstring. **You can accept just the next word** with `Ctrl+→` (or `Cmd+→` on Mac) — that walks the suggestion one word at a time.

**13.** Try it: Tab once for the full suggestion, or `Ctrl+→` repeatedly for word-by-word.

**14.** Esc to dismiss the rest if you don't want it.

---

## Drill 3: refusing a bad suggestion

**15.** Position the caret on a new line at the bottom of `routers/items.py`. Type:

```
def supercalifragilisticexpialidocious(
```

**16.** The autocomplete model will try to suggest something — likely garbage given the made-up name. Read the suggestion. **Do not accept.** Hit Esc.

**17.** Delete the line you typed.

**18. Lesson:** the model is a tool, not an oracle. Reading every suggestion before Tab is the habit.

---

## Drill 4: completing a test

**19.** Test your new route by curling it. First start the stack.

```
cd ~/docker-course/fastapi-app
docker compose up -d
sleep 3
```

**20.** Create an item to update.

```
curl -s -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name":"Original","price":1.0}' | python3 -m json.tool
```

Note the `id` in the response.

**21.** PUT to update it (substitute your id):

```
curl -s -X PUT http://localhost:8000/items/<id> \
  -H "Content-Type: application/json" \
  -d '{"name":"Updated","price":2.0}' | python3 -m json.tool
```

**22.** Read the response — `name` and `price` should be the new values, `id` unchanged.

---

## Drill 5: turn it off when you don't want it

**23.** Sometimes ghost text is distracting. Open the command palette (`Ctrl+Shift+P`) and run **"Continue: Toggle Autocomplete Enabled"**.

**24.** Ghost text now stops appearing. Re-run the same command to turn it back on.

---

## Drill 6: commit your new code

**25.** This is real work — commit it.

```
git switch -c add-update-item-route
git add routers/items.py
git commit -m "Add PUT /items/{item_id} update route"
```

**26.** We won't push this branch yet — Module 04's PR drill is sufficient for now. Switch back to `main` and clean up:

```
git switch main
git merge add-update-item-route
git branch -d add-update-item-route
```

**27.** Log it.

```
echo "Autocomplete used to write PUT /items route" >> ~/docker-
course/notes/notes.txt
```

---

## Watch out

- **Read every suggestion before Tab.** The 8B model is fast but limited. It will sometimes invent module names, miss imports, or hallucinate field names. Tab is a commitment.
- **Latency varies with file size.** Long files mean more context prefix to process — completions can take 1–2 seconds on big files. Patience.
- **Imports.** The autocomplete model often won't add `import` lines for symbols it uses. You'll fix imports manually or via Continue chat.
- **Ghost text only appears at the caret.** It won't suggest in the middle of an existing line — only when you stop typing in a position where new code makes sense.

---

## Checkpoint

```
grep -A 1 "@router.put" routers/items.py
```

Should show your new PUT route decorator.

```
docker compose up -d && sleep 3 && curl -s -X POST http://localhost:8000/items -H
"Content-Type: application/json" -d '{"name":"T","price":1}' | python3 -m json.tool
```

Should return a created item.

```
git log --oneline -1
```

Should show your "Add PUT /items/{item_id}" commit.

---

**Next up:** 07-inline-edit.md — Tell the model to edit highlighted code

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.